SELF LEARNING ACTIVITY

I. COMPARISON OF AGILE VS DEVOPS

1. Agile

Agile is an approach to building software through short, repeated cycles rather than delivering the entire product in one go. It places a premium on adaptability, ongoing input from the customer, teamwork, and the capacity to adjust when requirements shift. The overall project is broken down into smaller chunks known as iterations or sprints instead of being built as a single large release.

Agile is centred mainly on how software gets planned and built. Teams check in on their progress at regular intervals and use what they learn to sharpen the product in the iterations that follow.

A typical Agile cycle looks like this:

Requirements → Planning → Development → Testing → Review → Feedback → Next Iteration

Because Agile is built around iteration, requirements are free to evolve as the project progresses, shaped by customer input and shifting business needs.

2. DevOps

DevOps brings development and operations together as one practice, with the goal of making software delivery faster, more dependable, and more efficient. It encourages the two teams to work as one and relies heavily on automated pipelines covering integration, delivery, testing, deployment, and monitoring.

AWS describes DevOps as a blend of cultural values, working practices, and tooling that lets organisations ship applications and services quickly while keeping reliability high.

A typical DevOps flow looks like this:

Plan → Code → Build → Test → Release → Deploy → Operate → Monitor → Feedback

DevOps also places heavy emphasis on automation, continuous integration and delivery, ongoing monitoring, cross-team collaboration, and quick feedback loops.

3. Agile vs DevOps

Basis	**Agile**	**DevOps**
Meaning	A methodology for developing software	A set of practices, culture, and supporting tools
Main Focus	Building the software itself	Development, delivery, and operations together

Goal	Adapt quickly as requirements change	Ship software fast without sacrificing reliability
Teams	Developers, testers, product owners, etc.	Development, operations, testing, security, etc.
Development	Iterative and incremental	Continuous and automated
Deployment	Not the central concern	A primary focus
Automation	Applied where it's needed	Heavily emphasised throughout
Feedback	Largely from customers and stakeholders	Drawn from customers, systems, and operations
Work Cycle	Sprints or iterations	An ongoing, continuous lifecycle
Main Benefit	Flexibility and quicker development	Faster, more dependable delivery

Relationship Between Agile and DevOps

Rather than competing, Agile and DevOps work hand in hand.

Agile guides teams on what to build and how to develop it in stages, while DevOps takes over from there — building, testing, releasing, deploying, operating, and monitoring it smoothly.

For example:

Agile:

"The customer wants a new online payment feature."

DevOps:

"Let's build, test, deploy, monitor, and keep refining that feature efficiently."

II. CASE STUDY OF JIRA AND ASANA IN REAL PROJECTS

2.1 Case Study: JIRA

JIRA, built by Atlassian, is a widely used tool for tracking project work. Software development teams rely on it to manage issues, tasks, user stories, projects, and Agile workflows.

Real Project Example: Online Shopping Application

Take a company building an Online Shopping Application. The project covers features such as:

- User registration
- User login
- Product search
- Shopping cart
- Online payment
- Order tracking

The team can turn these requirements into user stories and tasks inside JIRA.

Example User Story

> *As a customer, I want to add products to my shopping cart so that I can purchase them later.*

This story can be broken down into tasks such as:

- Design the shopping cart interface
- Build the Add-to-Cart functionality
- Set up the cart's database operations
- Calculate the total price
- Test the shopping cart

These tasks get distributed among team members and tracked through statuses such as:

To Do → In Progress → Testing → Done

In this way, JIRA gives the team one central place to organise and follow project work — particularly useful for Agile teams running backlogs, user stories, sprints, and Scrum or Kanban workflows.

2.2 Case Study: Asana

Asana is a tool for managing projects and work more broadly, helping teams keep tasks, responsibilities, and deadlines organised.

Real Project Example: Mobile Application Launch

Take a company getting ready to launch a new mobile application.

The project could include activities such as:

- Market research
- UI/UX design
- Application development
- Testing
- Marketing
- Product launch

These activities become tasks in Asana, each assigned to a team member.

Project Task Management

Task	Responsible Team	Status
Market Research	Marketing Team	Completed
UI/UX Design	Design Team	In Progress
Application Development	Development Team	In Progress
Testing	QA Team	Not Started

Marketing Campaign	Marketing Team	Not Started

Through this, team members can see at a glance what still needs doing, who owns it, and how far along it is.

JIRA vs Asana

Feature	**JIRA**	**Asana**
Main Purpose	Tracking software and project work	Managing projects and work generally
Best Suited For	Software development teams	General and cross-functional teams
User Stories	Strongly supported	Handled as tasks
Scrum/Kanban	Strongly supported	Supports project-style workflows
Issue Tracking	Strong	Task-based
Sprint Management	Strong	Less central
Task Management	Yes	Yes
Main Strength	Agile software development	Coordinating tasks and projects

III. WRITING EFFECTIVE USER STORIES AND ACCEPTANCE CRITERIA

3.1 User Stories

A User Story lays out a requirement in plain terms, told from the point of view of the person who will actually use the system. It captures what the user wants and why that matters.

A common format for writing one is:

> *As a [type of user], I want [goal], so that [benefit].*

Example

> *As a customer, I want to add products to my shopping cart so that I can purchase multiple products together.*

This breaks down into:

- User: Customer
- Requirement: Add products to cart
- Benefit: Purchase multiple products together

Characteristics of an Effective User Story

A good user story should be:

1. Clear – easy to follow and understand.
2. Concise – limited to what's necessary.

3. User-focused – written around the user's actual need.
4. Valuable – delivers a real benefit.
5. Testable – it should be possible to check whether it's been met.
6. Independent – not overly reliant on other stories.
7. Prioritized – the more important requirements get addressed first.

3.2 Acceptance Criteria

Acceptance criteria spell out the specific conditions a user story must meet before it can be marked complete and accepted.

They give the customer, product owner, developer, and tester a shared, unambiguous picture of what the feature should do.

Example User Story

> *As a customer, I want to add products to my shopping cart so that I can purchase them later.*

Acceptance Criteria

1. The customer can add an available product to the cart.
2. The chosen product then shows up in the cart.
3. The customer can increase or decrease the quantity.
4. The cart shows the correct total price.
5. The customer can remove a product from the cart.
6. An unavailable product cannot be added to the cart.

Acceptance criteria are what turn a user story into something specific, measurable, and testable.

IV. ADVANCED REQUIREMENT ELICITATION TECHNIQUES

Requirement elicitation is the process of gathering, understanding, and documenting what stakeholders need and expect.

Two notable advanced techniques for this are interviews and ethnography.

4.1 Interviews

An interview is a face-to-face technique where a requirements analyst questions stakeholders directly to learn about their needs, expectations, problems, and priorities.

Types of Interviews

i. Structured Interview: A fixed list of questions is worked out ahead of the interview.

ii. Unstructured Interview: The conversation stays open-ended, letting the stakeholder describe their requirements in their own way.

iii. Semi-structured Interview: The analyst comes with key questions prepared but can branch into others as the conversation develops.

Interview Process

1. Identify the stakeholders.
2. Prepare relevant questions.
3. Conduct the interview.
4. Record the important information.
5. Analyze the collected information.
6. Validate the requirements with stakeholders.
7. Document the final requirements.

Example

For an online banking system, an analyst might interview both customers and bank staff to draw out requirements around:

- Login
- Fund transfer
- Transaction history
- Notifications
- Security
- Account management

Advantages

- Yields detailed information.
- Lets the analyst clarify things on the spot.
- Surfaces requirements that might otherwise stay hidden.
- Gives direct feedback straight from the stakeholder.

Limitations

- Can eat up a lot of time.
- Stakeholders may give biased or incomplete answers.
- The requirements gathered are only as good as the questions asked.

4.2 Ethnography

Ethnography involves the analyst observing users directly in their actual work setting to see how they really carry out their tasks.

Where interviews rely on users describing what they do, ethnography lets the analyst watch what users actually do in real situations.

Example

Say a software company is building a system for hospital nurses. Rather than relying only on interviews, the analyst watches the nurses' day-to-day work.

Through observation, the analyst might find that nurses:

- Need patient information available at a glance.
- Move constantly between departments.
- Have very little spare time for data entry.
- Need alerts the moment something important happens.

These are the kinds of practical requirements that might never come up in an interview.

Advantages

- Captures how users genuinely behave.
- Surfaces hidden requirements.
- Builds a real understanding of day-to-day workflows.
- Well suited to complex work environments.

Limitations

- Can be time-intensive.
- Users may act differently once they know they're being watched.
- Depends on getting access to the real work environment.
- The volume of observational data collected can be hard to analyse.

V. REQUIREMENT TRACEABILITY MATRIX (RTM)

5.1 Definition

A Requirement Traceability Matrix (RTM) is a table or document that maps requirements to other project artifacts — design components, development activities, and test cases.

Its core purpose is making sure every requirement actually gets implemented and tested.

The underlying relationship is:

Requirement → Design → Development → Test Case → Test Result

5.2 Example of RTM

For an Online Shopping System:

Requirement ID	Requirement	Module	Test Case ID	Status
REQ-01	User registration	Registration	TC-01	Completed
REQ-02	User login	Login	TC-02	Completed
REQ-03	Product search	Search	TC-03	Completed
REQ-04	Add product to cart	Cart	TC-04	In Progress
REQ-05	Online payment	Payment	TC-05	Pending

Importance of RTM

RTM helps to:

- Confirm every requirement is accounted for.
- Link requirements to the test cases that verify them.
- Flag missing requirements or gaps in testing.
- Keep track of requirements across the project.
- Manage changes to requirements as they arise.
- Raise overall software quality.
- Confirm the finished product meets the specified requirements.

VI. TOOLS FOR REQUIREMENT MANAGEMENT

Requirement management tools help organisations capture, organise, prioritise, track, trace, and manage changes to requirements across the software development lifecycle.

Examples

- IBM DOORS
- JIRA
- Azure DevOps
- Jama Connect
- Polarion

6.1 IBM DOORS

IBM Engineering Requirements Management DOORS is a tool built for capturing, organising, analysing, tracing, and managing requirements. It's designed to help teams keep track of how requirements relate to the rest of the project's information.

Major Features of IBM DOORS

1. Requirement Management: Teams can create, organise, store, and manage the project's requirements.

2. Requirement Traceability: Requirements can be linked to related requirements and other project artifacts, making it easier to trace how each one is implemented and verified.

3. Change Management: Changes to requirements can be tracked, so teams understand what a modification will affect.

4. Requirement Analysis: Teams can review requirements and how they relate to one another.

5. Collaboration: Different stakeholders can work with the requirements throughout the project.

Example

Take an automotive software project with requirements spanning:

- Vehicle safety
- Braking systems
- Sensors
- Performance
- Security

A project like this can involve a huge number of requirements. IBM DOORS helps organise them and keep traceability intact between requirements, development work, and testing.

When a requirement changes, its related information can be traced, helping the team see what else in the project might be affected.

Advantages of IBM DOORS

- Centralised requirement management
- Requirement traceability
- Change tracking
- Better-organised requirements
- Stronger collaboration
- Useful for large, complex projects